WEEK 4 · LESSON 20 OF 24 · HANDOUT

Fine-Tuning, Prompting or RAG? And LoRA from Scratch

Study time: **about 60 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=hdHKCjhF68U — open the lesson online to watch inside the course.

🎓 **Go deeper:** posit::conf 2023: A hacker's guide to open source LLMs (Jeremy Howard) — https://www.youtube.com/watch?v=sYliwvml9Es

🎓 **Go deeper:** Andrej Karpathy: Deep Dive into LLMs like ChatGPT — https://www.youtube.com/watch?v=7xTGNNLPyMI

Your goal in this lesson: Choose between prompting, RAG and fine-tuning for a task, and understand parameter-efficient fine-tuning by building LoRA yourself in PyTorch.

Three ways to adapt a model

Approach	What changes	Good for	Weak at
Prompting	Only the input	Most tasks; fast to try	Very long rules, many examples
RAG	The input gets retrieved documents	Facts, documents, knowledge that changes	Style, format, new skills
Fine-tuning	The model's weights	Consistent style or format, a narrow task at large scale, a smaller cheaper model	Adding facts (it may still invent them), keeping up to date

A practical order: **prompt first → add RAG for knowledge → fine-tune only when you have hundreds of good examples and a measured reason.** Fine-tuning is the most work: data, training, evaluation and hosting.

Training data for fine-tuning a chat model

Fine-tuning data is a list of example conversations showing exactly the behaviour you want, often saved as JSON Lines (one JSON object per line):

```
import json

examples = [
    {"messages": [{"role": "user", "content": "My internet is slow since morning."},
                  {"role": "assistant", "content": "CATEGORY: network | PRIORITY: normal"}]},
    {"messages": [{"role": "user", "content": "Someone stole my phone, block my SIM now!"},
                  {"role": "assistant", "content": "CATEGORY: account | PRIORITY: urgent"}]},
]
with open("train.jsonl", "w") as f:
    for ex in examples:
        f.write(json.dumps(ex) + "\n")
print(open("train.jsonl").read())
```

Quality matters more than quantity: a few hundred **correct, consistent, varied** examples beat thousands of messy ones. Keep a separate test set, exactly as in Machine Learning Fundamentals.

Why full fine-tuning is expensive

A 7-billion-parameter model has 7 billion numbers to update. Training them all needs the weights, their gradients and optimizer state in memory — often more than 100 GB. **Parameter-efficient fine-tuning (PEFT)** trains only a tiny number of new parameters instead. The most popular method is **LoRA (Low-Rank Adaptation)**.

The LoRA idea

For a frozen weight matrix W (size out × in), LoRA learns two small matrices **B** (out × r) and **A** (r × in) with a small rank r, and uses:

> *output = W·x + (alpha ÷ r) · B·A·x*

With in = out = 4,096 and r = 8, the full matrix has 16.8 million numbers but A and B together only 65,536 — **0.4%**. B starts at zero, so training starts exactly from the original model.

Build it: a base model and a new task

We train a base network on handwritten digits, then adapt it to a **new task**: the same digits mirrored left-to-right, with only 100 examples.

```
import copy
import torch
import torch.nn as nn
from sklearn.datasets import load_digits

torch.manual_seed(0)
digits = load_digits()
X = torch.tensor(digits.data / 16, dtype=torch.float32)
y = torch.tensor(digits.target)
X_mirror = torch.flip(X.view(-1, 8, 8), dims=[2]).reshape(-1, 64)
perm = torch.randperm(len(X))
train_idx, test_idx = perm[:1200], perm[1200:]

def train(model, X, y, epochs=300, lr=1e-2):
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.Adam(params, lr=lr)
    for _ in range(epochs):
        opt.zero_grad()
        loss = nn.functional.cross_entropy(model(X), y)
        loss.backward()
        opt.step()
    return model

def accuracy(model, X, y):
    with torch.no_grad():
        return (model(X).argmax(1) == y).float().mean().item()

base = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 128), nn.ReLU(),
nn.Linear(128, 10))
train(base, X[train_idx], y[train_idx])
print(f"base model  original digits: {accuracy(base, X[test_idx], y[test_idx]):.1%}"
      f"   mirrored digits: {accuracy(base, X_mirror[test_idx], y[test_idx]):.1%}")
```

The base model is excellent on normal digits but much worse on mirrored ones — a new task it never saw.

LoRA layer from scratch

```
class LoRALinear(nn.Module):
    def __init__(self, base_layer, r=4, alpha=8):
        super().__init__()
        self.base = base_layer
        for p in self.base.parameters():
            p.requires_grad = False                        # freeze the original weights
        self.A = nn.Parameter(torch.randn(r, base_layer.in_features) * 0.01)
        self.B = nn.Parameter(torch.zeros(base_layer.out_features, r))   # zero: start =
original model
        self.scale = alpha / r
        self.enabled = True

    def forward(self, x):
        out = self.base(x)
        if self.enabled:
            out = out + (x @ self.A.T @ self.B.T) * self.scale
        return out

def add_lora(model, r=4):
    model = copy.deepcopy(model)
    for i, layer in enumerate(model):
        if isinstance(layer, nn.Linear):
            model[i] = LoRALinear(layer, r=r)
    return model

count = lambda m: sum(p.numel() for p in m.parameters() if p.requires_grad)
```

Compare full fine-tuning and LoRA

```
few = train_idx[:100]                                      # only 100 mirrored examples

full = train(copy.deepcopy(base), X_mirror[few], y[few], epochs=200)
lora = train(add_lora(base, r=4), X_mirror[few], y[few], epochs=200)

print(f"{'method':12s} {'trainable':>10s} {'mirrored':>9s} {'original':>9s}")
for name, m in [("base", base), ("full", full), ("LoRA r=4", lora)]:
    print(f"{name:12s} {count(m) if name != 'base' else 0:>10,} "
          f"{accuracy(m, X_mirror[test_idx], y[test_idx]):>9.1%} {accuracy(m, X[test_idx],
y[test_idx]):>9.1%}")

for layer in lora:
    if isinstance(layer, LoRALinear):
        layer.enabled = False                              # switch the adapter off
print(f"LoRA switched off -> original digits: {accuracy(lora, X[test_idx], y[test_idx]):.1%}")
```

What to notice in your results:

- **Trainable parameters:** full fine-tuning updates all 26,122; LoRA trains only 2,344 — about one eleventh here, and far less (under 1%) in large models.
- **New task:** both lift mirrored accuracy from about **46%** to about **81%** with just 100 examples.
- **Forgetting:** after full fine-tuning, accuracy on normal digits falls from 97% to about **69%** — the old skill is damaged for good ("catastrophic forgetting"). With the LoRA adapter switched on, normal digits also suffer, but switching it off gives back **exactly 97%**, because the original weights were never changed.

This is why companies can keep one base model and many small LoRA adapters — one per customer or task — and swap them in and out.

Fine-tuning real LLMs

- For open models (next lesson), the Hugging Face **PEFT** library adds LoRA to chosen layers in a few lines; **QLoRA** combines LoRA with a 4-bit quantized base model so a 7B model can be fine-tuned on one free Colab GPU.
- Many API providers also offer fine-tuning services for some of their models: you upload JSONL examples and get a private model.
- Always compare the fine-tuned model with a good prompt + RAG baseline on the **same test set**. If it is not clearly better, do not fine-tune.

Key terms

- **Fine-tuning** — further training of a pre-trained model on your examples.
- **PEFT** — parameter-efficient fine-tuning: train only a few new parameters.
- **LoRA** — learns a low-rank update B·A added to frozen weights.
- **Rank (r)** — the inner size of A and B; small r = few parameters.
- **Catastrophic forgetting** — losing old skills after training on a new task.
- **QLoRA** — LoRA on top of a 4-bit quantized base model.

Remember this: Prompt first, use RAG for knowledge and fine-tune for behaviour; LoRA makes fine-tuning cheap by training a tiny low-rank update while the original model stays frozen and recoverable.

Name: Date:

Activity

Task 1

Try `r=1`, `r=2` and `r=16`. How do trainable parameters and mirrored accuracy change?

Task 2

Train the LoRA model with only 30 mirrored examples. Does it still help?

Task 3

"Merge" the adapter: compute `W + scale * B @ A` for one layer and check it gives the same output as the LoRA layer.

Task 4

For three tasks from your work, decide: prompting, RAG or fine-tuning? Write one sentence of reasoning for each.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Which approach is best for answering from documents that change every week?
   - A. RAG
   - B. None
   - C. Training from scratch
   - D. Fine-tuning
2. When is fine-tuning a good choice?
   - A. To add last week's prices
   - B. Before trying any prompt
   - C. For consistent style or format on a narrow task, with hundreds of good examples
   - D. When you have five examples
3. What does LoRA train?
   - A. All the weights
   - B. Only the tokenizer
   - C. The embeddings only
   - D. Two small low-rank matrices added to frozen weights
4. Why does matrix B start at zero?
   - A. To save memory
   - B. To make training slower
   - C. So training starts exactly from the original model
   - D. It is random
5. What is catastrophic forgetting?
   - A. A memory error
   - B. Forgetting the prompt
   - C. Losing old skills after training on a new task
   - D. Losing the API key
6. What is QLoRA?
   - A. A chat app
   - B. A tokenizer
   - C. A vector database
   - D. LoRA on top of a 4-bit quantized base model

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.